Complete OPERATING SYSTEMS

08 April 2026 01:17

-- TOPICS COVERED IN THIS VIDEO:

1. Introduction
- OS basics → role, goals, functions

2. Architecture
- OS structure → layers, kernel, system calls

3. Types of OS
- OS types → batch, multitasking, RTOS, etc.

4. Process
- Process → definition, states, PCB, context switch

5. Threads
- Threads → types, models, benefits

6. Scheduling
- CPU scheduling → queues, criteria, algorithms

7. Synchronization
- Critical section → mutex, semaphore, problems

8. Deadlocks
- Deadlock → conditions, prevention, banker's

9. Memory Management
- Memory → allocation, paging, segmentation

10. Virtual Memory
- VM → demand paging, page fault, replacement

11. File System
- Files → structure, allocation, directories

12. I/O Management
- I/O → techniques, buffering, DMA

13. Disk Scheduling
- Disk → seek optimization algorithms

14. Storage & Cache
- Cache → locality, hit/miss, policies

15. Security
- Protection → auth, access control, threats

16. Advanced Topics
- Modern OS → virtualization, containers, CFS

17. OS Internals
- Boot → kernel, interrupts, modes

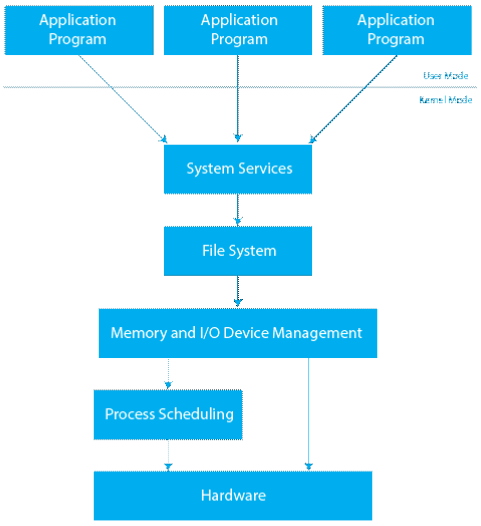
18. Interview Questions
- Most asked → OS core concepts revision

SECTION 1 - INTRODUCTION TO OPERATING SYSTEMS

Definition

An **Operating System (OS)** is system software that acts as an **intermediary between hardware and the user/application software**. It manages hardware resources and provides a platform for programs to run.

"An OS is a resource allocator and a control program."



Why Do We Need an OS?

- Without OS, every program would need to directly manage CPU, RAM, Disk, I/O
- OS abstracts hardware complexity
- Provides security, multi-tasking, memory management, file handling

Goals of an OS

Primary Goal --> Convenience (easy to use)

- OS abstracts hardware complexity
- Provides security, multi-tasking, memory management, file handling

Goals of an OS

Primary Goal --> Convenience (easy to use)
Secondary Goal --> Efficiency (use hardware well)

Functions of OS

```
+---------------------------+
| Process Management        | --> Create, schedule, terminate processes
| Memory Management         | --> Allocate / deallocate RAM
| File System Management    | --> Files, directories, permissions
| I/O Management            | --> Device drivers, buffering
| Security & Protection     | --> Auth, access control
| Network Management        | --> Protocols, sockets
+---------------------------+
```

SECTION 2 - ARCHITECTURE OF AN OPERATING SYSTEM

Layered Architecture

```
+--------------------------------+
|        User Programs           | <-- Layer 5 (Top)
+--------------------------------+
|     System Utilities           | <-- Layer 4
+--------------------------------+
|    Operating System API        | <-- Layer 3
+--------------------------------+
|        Kernel                  | <-- Layer 2 (Core)
+--------------------------------+
|        Hardware                | <-- Layer 1 (Bottom)
+--------------------------------+
```

Kernel

- The **core** of the OS
- Always resident in memory
- Handles: CPU scheduling, memory, I/O, inter-process communication (IPC)

Types of Kernel

Monolithic Kernel --> All OS services in one large block (Linux, Unix)
Fast but less modular
Microkernel --> Minimal kernel; services run in user space (Mach, QNX)
Slower (message passing) but more stable/secure
Hybrid Kernel --> Mix of both (Windows NT, macOS XNU)
Balance of performance and modularity
Exokernel --> Extremely minimal; gives programs direct hardware access
Used in research OSes

System Calls

- **Definition:** Interface between a user program and the OS. When an app needs a privileged operation, it makes a **system call**.
- Examples: fork(), exec(), read(), write(), open(), close(), exit()

App --> System Call --> Kernel Mode --> Hardware
(trap/interrupt)

Types of System Calls

1. Process Control --> fork, exit, wait, exec
2. File Management --> open, read, write, close
3. Device Management --> ioctl, read, write
4. Information --> getpid, alarm, sleep
5. Communication --> pipe, socket, shmget

SECTION 3 - TYPES OF OPERATING SYSTEMS

1. Batch OS

- Jobs collected, grouped into batches, executed without user interaction

SECTION 3 - TYPES OF OPERATING SYSTEMS

1. Batch OS

- Jobs collected, grouped into batches, executed without user interaction
- No direct user-CPU interaction
- Example: IBM OS/360

Jobs --> Job Queue --> Batch Monitor --> CPU --> Output

2. Multiprogramming OS

- Multiple programs loaded in memory simultaneously
- When one process waits (I/O), CPU switches to another
- Increases CPU utilization

RAM: [P1][P2][P3] --> CPU picks one at a time

3. Multitasking / Time-Sharing OS

- Extension of multiprogramming
- CPU switches rapidly between processes giving illusion of parallelism
- Each process gets a **time quantum** (e.g., 10ms)
- Example: UNIX, Windows

4. Multiprocessing OS

- Multiple CPUs in one system
- True parallelism (multiple processes run at same time)
- Example: Server OS, Linux SMP

5. Real-Time OS (RTOS)

- Strict timing constraints
- **Hard RTOS:** Must meet deadline always (Airbag, pacemaker)
- **Soft RTOS:** Deadline preferred but not critical (Video streaming)
- Examples: VxWorks, FreeRTOS

6. Distributed OS

- Multiple computers networked together appear as one system
- Example: Google's systems, Hadoop clusters

7. Embedded OS

- Designed for specific hardware (smart TVs, microwaves, routers)
- Example: Android (mobile), iOS, FreeRTOS

SECTION 4 - PROCESS MANAGEMENT

What is a Process?

- A **process** is a **program in execution**
- It is an active entity (program is passive -- just a file)
- A process has its own memory space, state, and resources

Process vs Program

Program	Process
Passive (on disk)	Active (in memory)
Static	Dynamic
One copy on disk	Multiple instances possible
No resources owned	Owns CPU, RAM, files

Process Components (PCB)

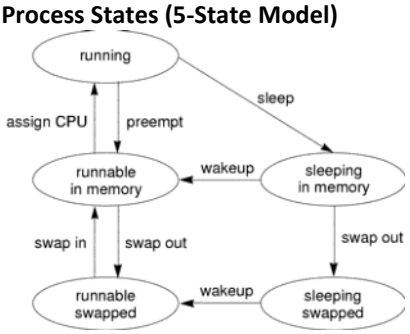
Each process is represented by a **Process Control Block (PCB)**:

```
+-------------------------+
| Process ID (PID)        |
| Process State           |
| Program Counter (PC)    |
| CPU Registers           |
| Memory Management Info  |
| I/O Status Info         |
| Accounting Info         |
+-------------------------+
```

Process States (5-State Model)



Process States (5-State Model)




```
          admit
NEW  ---------> READY <--------+
          |         |
      (dispatch)      | (I/O complete /
          v         |  event occurs)
        RUNNING ------> WAITING
          |
      (exit/terminate)
          v
       TERMINATED
```

- **New:** Process is being created
- **Ready:** Process is waiting to be assigned to CPU
- **Running:** Instructions are being executed
- **Waiting (Blocked):** Process waiting for I/O or event
- **Terminated:** Process has finished

Context Switching

- When CPU switches from one process to another
- Current process state saved in PCB
- New process state loaded from its PCB
- **Overhead:** Context switching wastes CPU time (pure overhead)

P1 Running --> save P1 PCB --> load P2 PCB --> P2 Running

SECTION 5 - THREADS

What is a Thread?

- A **thread** is the smallest unit of CPU execution
- A process can have multiple threads (lightweight processes)
- Threads of same process **share**: code, data, files
- Threads have their own: **stack, registers, PC**

Process vs Thread

Process	Thread
Heavy-weight	Light-weight
Separate memory space	Shared memory space
Slow to create/switch	Fast to create/switch
IPC needed to communicate	Direct memory sharing
Crash isolates	One thread crash can kill all

Types of Threads

User-Level Threads (ULT)
--> Managed by user-space library (no kernel awareness)
--> Fast, but one blocking call blocks all threads
--> Example: POSIX pthreads in older apps

Kernel-Level Threads (KLT)
--> Managed by OS kernel
--> True parallelism on multicore
--> Slower to create but each can be independently scheduled
--> Example: Windows threads, Linux pthreads today

Multithreading Models

Many-to-One --> Many user threads mapped to one kernel thread
(Blocking issue)

--> Example: Windows threads, Linux pthreads today

Multithreading Models

Many-to-One --> Many user threads mapped to one kernel thread
(Blocking issue)
One-to-One --> Each user thread maps to one kernel thread
(Used by Linux, Windows)
Many-to-Many --> Many user threads map to many kernel threads
(Best of both worlds)

Benefits of Multithreading

- **Responsiveness** - UI stays active while background work runs
- **Resource sharing** - Threads share process memory
- **Economy** - Creating threads is cheaper than processes
- **Scalability** - Utilize multiple cores

SECTION 6 - CPU SCHEDULING

Why Scheduling?

- Multiprogramming requires deciding which process runs on CPU next
- Goal: Maximize CPU utilization and minimize waiting time

Scheduling Queues

Job Queue --> All processes in system
Ready Queue --> Processes in RAM ready to execute
Device Queue --> Processes waiting for I/O device

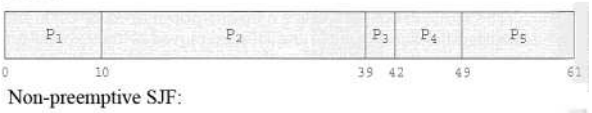
Types of Schedulers

Long-Term (Job Scheduler)
--> Decides which programs get loaded into RAM
--> Controls degree of multiprogramming
--> Infrequent (seconds/minutes)
Short-Term (CPU Scheduler)
--> Decides which ready process gets the CPU
--> Very frequent (milliseconds)
Medium-Term (Swapper)
--> Swaps processes in/out of memory
--> Used in time-sharing systems

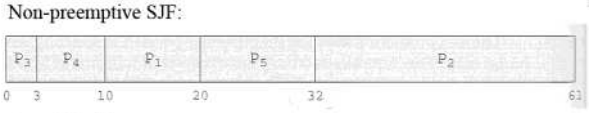
Scheduling Criteria

CPU Utilization --> Keep CPU as busy as possible (ideally 40-90%)
Throughput --> Number of processes completed per time unit
Turnaround Time --> Submission to completion time (TAT = CT - AT)
Waiting Time --> Time spent in ready queue (WT = TAT - BT)
Response Time --> Time from submission to first response

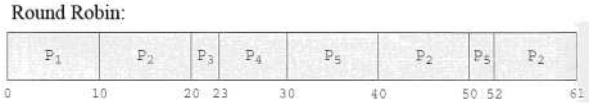
FCFS:

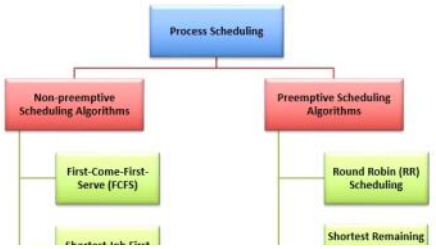


Non-preemptive SJF:

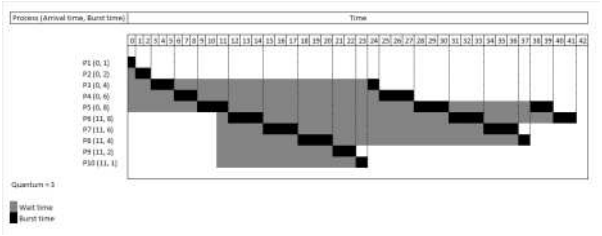


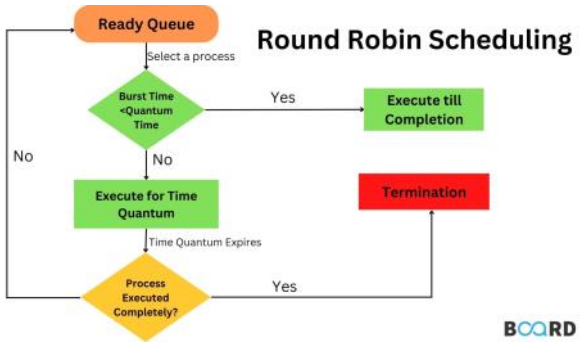
Round Robin:











SCHEDULING ALGORITHMS

1. FCFS - First Come First Served

- Non-preemptive
- Process that arrives first gets CPU first
- Simple but **Convoy Effect** problem (short process waits behind long one)

Example:

Process AT BT
P1 0 5
P2 1 3
P3 2 2

Gantt Chart:
| P1(0-5) | P2(5-8) | P3(8-10) |
WT: P1=0, P2=4, P3=6 --> Avg WT = 10/3 = 3.33

2. SJF - Shortest Job First

- Non-preemptive
- Process with shortest burst time gets CPU
- Optimal in terms of average waiting time
- **Problem:** Starvation (long processes may never run), Need to know burst time in advance

Example:

Process AT BT
P1 0 6
P2 0 2
P3 0 4

Gantt Chart:
| P2(0-2) | P3(2-6) | P1(6-12) |
WT: P1=6, P2=0, P3=2 --> Avg WT = 8/3 = 2.67

3. SRTF - Shortest Remaining Time First (Preemptive SJF)

- Preemptive version of SJF
- If a new process arrives with shorter burst time than remaining time of current, CPU preempts
- Optimal but high context switching overhead

4. Round Robin (RR)

- Each process gets a fixed **time quantum (q)**
- After q expires, process preempted and goes to end of ready queue
- Most used in **time-sharing systems**
- **Key:** If q too large --> FCFS. If q too small --> too many context switches.

Example (q=2):

Process AT BT
P1 0 4
P2 0 3
P3 0 2

Gantt Chart:
| P1(0-2) | P2(2-4) | P3(4-6) | P1(6-8) | P2(8-9) |
P1 WT=4, P2 WT=4, P3 WT=2 --> Avg = 3.33

5. Priority Scheduling

- Each process assigned a priority number
- CPU given to highest priority (lower number = higher priority in most systems)

P1 WT=4, P2 WT=4, P3 WT=2 --> Avg = 3.33

5. Priority Scheduling

- Each process assigned a priority number
- CPU given to highest priority (lower number = higher priority in most systems)
- Can be preemptive or non-preemptive
- **Problem:** Starvation
- **Solution:** Aging -- gradually increase priority of waiting processes

6. Multilevel Queue Scheduling

- Ready queue divided into multiple queues (e.g., system, interactive, batch)
- Each queue has its own scheduling algorithm
- No movement between queues

7. Multilevel Feedback Queue

- Like multilevel queue but **processes can move between queues**
- Most flexible and complex
- Used in most modern OS (like Linux CFS concept)

Scheduling Comparison Table

Algorithm	Preemptive?	Starvation?	Complexity
FCFS	No	No	Simple
SJF	No	Yes	Medium
SRTF	Yes	Yes	Medium
RR	Yes	No	Medium
Priority	Both	Yes	Medium
MLQ	Both	Yes	Complex
MLFQ	Both	No	High

SECTION 7 - PROCESS SYNCHRONIZATION

The Critical Section Problem

- When multiple processes access **shared data** concurrently, we get **Race Condition**
- **Critical Section** = Code segment that accesses shared resources

Requirements for Critical Section Solution

1. Mutual Exclusion --> Only one process in CS at a time
2. Progress --> If CS is empty, a waiting process should enter
3. Bounded Waiting --> A process must enter CS within finite time (no starvation)

Peterson's Solution (Software Solution for 2 processes)

```
// For Process Pi
flag[i] = true;
turn = j;
while (flag[j] && turn == j);
// CRITICAL SECTION
flag[i] = false;
```

- Works for 2 processes only
- Satisfies all 3 conditions

LOCKS AND SYNCHRONIZATION PRIMITIVES

Mutex (Mutual Exclusion Lock)

- Binary lock: 0 = unlocked, 1 = locked
- Only the thread that **locks** can **unlock**
- **Busy waiting / Spinlock** problem

```
lock()
  while (mutex == 1);   // busy wait
  mutex = 1;
unlock()
  mutex = 0;
```

Semaphore

- **Definition:** Integer variable accessed via atomic wait() and signal() operations
- More powerful than mutex

```
wait(S):          signal(S):
  S--;              S++;
  if S < 0:         if S <= 0:
    block             wake up a process
```

Types:

Binary Semaphore --> Value 0 or 1 (like mutex, but any thread can signal)
Counting Semaphore --> Value >= 0 (controls pool of resources)

Example - Counting Semaphore:

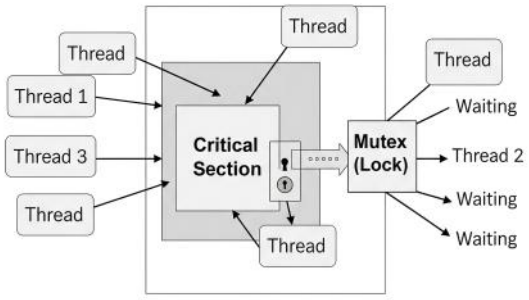
5 printers available --> Semaphore S = 5

Binary Semaphore --> Value 0 or 1 (like mutex, but any thread can signal)
Counting Semaphore --> Value >= 0 (controls pool of resources)

Example - Counting Semaphore:

5 printers available --> Semaphore S = 5
Each print job: wait(S) -- S becomes 4,3,2,1,0
After printing: signal(S) -- S goes back up
If S=0, next process blocks until one printer freed

Mutex and Locking



CLASSICAL SYNCHRONIZATION PROBLEMS

1. Producer-Consumer (Bounded Buffer)

Producer adds items to buffer
Consumer removes items from buffer
Buffer has fixed size N

Semaphores:
mutex = 1 (mutual exclusion)
full = 0 (items in buffer)
empty = N (empty slots)

Producer:	Consumer:
wait(empty)	wait(full)
wait(mutex)	wait(mutex)
add item	remove item
signal(mutex)	signal(mutex)
signal(full)	signal(empty)

2. Readers-Writers Problem

- Multiple readers can read simultaneously
- Writer needs exclusive access
- **First variation:** No reader waits unless writer has lock
- **Second variation:** No writer waits once ready (writer priority)

3. Dining Philosophers Problem

5 philosophers around a table
5 chopsticks (forks) between them
Each needs 2 chopsticks to eat
Problem: Deadlock possible if all pick left chopstick simultaneously

Solution:
- Allow only 4 philosophers to sit
- Odd-numbered pick left first, even pick right first
- Use semaphore array

SECTION 8 - DEADLOCKS

Definition

A **deadlock** is a situation where a **set of processes are blocked** because each process is holding a

SECTION 8 - DEADLOCKS

Definition

A **deadlock** is a situation where a **set of processes are blocked** because each process is holding a resource and **waiting for another** resource held by another process in the set.

Coffman's 4 Necessary Conditions (ALL must hold for deadlock)

1. Mutual Exclusion --> Resource held by only one process at a time
2. Hold and Wait --> Process holds resource while waiting for more
3. No Preemption --> Resource cannot be forcibly taken
4. Circular Wait --> P1 waits for P2, P2 waits for P3, P3 waits for P1

TRICK: Remember as "MHNC" or "MH-NP-CW"

Resource Allocation Graph (RAG)

Process --> Resource (Request Edge)
Resource --> Process (Assignment Edge)
Deadlock detection:
- If RAG has a cycle with single-instance resources --> Deadlock
- With multi-instance resources --> Cycle necessary but not sufficient

Deadlock Handling Strategies

1. Prevention

- Remove one of the 4 conditions:

No Mutual Exclusion --> Make resources shareable (not always possible)
No Hold & Wait --> Request all resources at once before starting
Allow Preemption --> If resource needed, preempt from waiting process
No Circular Wait --> Assign ordering to resource types; request in order

2. Avoidance - Banker's Algorithm

- **Definition:** OS checks in advance if allocating a resource will leave the system in a **safe state**
- **Safe State:** There exists a sequence of processes (safe sequence) where all can complete

Key Data Structures:
- Available[m] --> Resources currently available
- Max[n][m] --> Max demand of each process
- Allocation[n][m] --> Resources allocated to each process
- Need[n][m] --> Max - Allocation (remaining need)

Safe State Check:

```
Work = Available
For each process Pi not finished:
  if Need[i] <= Work:
    Work = Work + Allocation[i]
    mark Pi as finished
If all processes finished --> Safe State!
```

Example:

Resources: A=3, B=3, C=2 (Available)

Process	Alloc	Need
P0	0,1,0	7,4,3
P1	2,0,0	1,2,2
P2	3,0,2	6,0,0
P3	2,1,1	0,1,1
P4	0,0,2	4,3,1

Safe sequence: P1 --> P3 --> P4 --> P0 --> P2

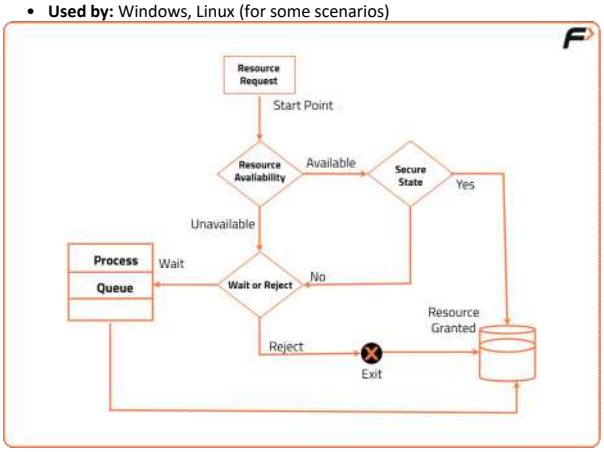
3. Detection and Recovery

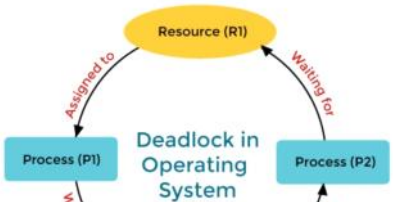
- Allow deadlock to occur, detect it, then recover
- **Detection:** Run deadlock detection algorithm periodically
- **Recovery Options:**

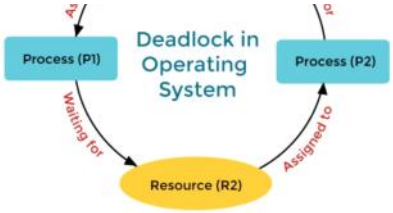
- Process Termination --> Kill all deadlocked processes
  Kill one at a time (expensive but careful)
- Resource Preemption --> Take resource from a process
  Rollback and restart victim process

4. Ignorance (Ostrich Algorithm)

- Pretend deadlocks don't happen
- Used when deadlocks are rare and cost of prevention > cost of occasional reboot
- **Used by:** Windows, Linux (for some scenarios)







SECTION 9 - MEMORY MANAGEMENT

Why Memory Management?

- Multiple processes in RAM simultaneously
- Need to allocate and deallocate memory efficiently
- Provide isolation (process A can't access process B's memory)

Memory Hierarchy

Registers --> Fastest, smallest (inside CPU)
Cache --> L1, L2, L3 (nanoseconds)
RAM/DRAM --> Main memory (microseconds)
SSD/HDD --> Secondary storage (milliseconds)
Network --> Slowest

Address Binding

Compile Time --> Absolute addresses (if location known at compile time)
Load Time --> Relocatable code; addresses assigned when loaded
Execution Time --> Addresses translated at runtime (needs MMU)

Logical vs Physical Address

Logical Address --> Generated by CPU (virtual address)
Physical Address --> Actual RAM location
MMU (Memory Management Unit) maps logical --> physical

CONTIGUOUS MEMORY ALLOCATION

Fixed Partitioning

- RAM divided into fixed-size partitions
- **Internal Fragmentation:** Allocated partition larger than needed

Partition: 100MB | Process needs: 80MB | Waste: 20MB --> Internal Frag

Dynamic Partitioning

- Partitions created as needed (exact size)
- **External Fragmentation:** Free memory scattered, total free > needed but not contiguous

Fragmentation

Internal Fragmentation --> Wasted space INSIDE allocated partition
External Fragmentation --> Wasted space OUTSIDE allocations (scattered free blocks)

Allocation Strategies

First Fit --> Allocate first hole that is big enough (Fast)
Best Fit --> Allocate smallest hole that fits (Minimizes waste, slow)
Worst Fit --> Allocate largest hole (Leaves large remainders, least used)
Next Fit --> Like First Fit but starts from last allocation point

Compaction

- Move all processes together to one end, merge free memory
- Solves external fragmentation but **very expensive** (requires process relocation)

PAGING

Definition

- Physical memory divided into fixed-size **frames**
- Logical memory divided into same-size **pages**
- OS maintains a **Page Table** for each process: maps page number to frame number

Logical Address = [Page Number | Offset]
Physical Address = [Frame Number | Offset]
Physical Address = Frame_Base_Address + Offset

Page Table

Page	Frame	Valid Bit

Physical Address = Frame_Base_Address + Offset

Page Table

Page	Frame	Valid Bit
0	3	1
1	7	1
2	-	0 (not in memory)
3	1	1

Address Translation Example

Page Size = 4KB = 2^12
Logical Address: 0x3ABC
Page Number = 0x3ABC >> 12 = 3
Offset = 0xABC
Page Table: Page 3 --> Frame 5
Physical Address: (5 * 4096) + 0xABC = 0x5ABC

TLB (Translation Lookaside Buffer)

- **Hardware cache** for page table entries
- Makes address translation fast
- **TLB Hit:** Translation found in TLB (fast)
- **TLB Miss:** Must look up page table in RAM (slow)
- **Effective Access Time (EAT):**

EAT = hit_ratio * (TLB_time + mem_time) + (1 - hit_ratio) * (TLB_time + 2*mem_time)

Multi-level Paging

- With large address spaces, page table itself becomes too large
- Split page table into multiple levels

32-bit address with 2-level paging:
[10-bit dir | 10-bit page | 12-bit offset]

Inverted Page Table

- One entry per physical frame (not per logical page)
- Reduces page table size
- Slower lookup (search by PID + page number)

SEGMENTATION

Definition

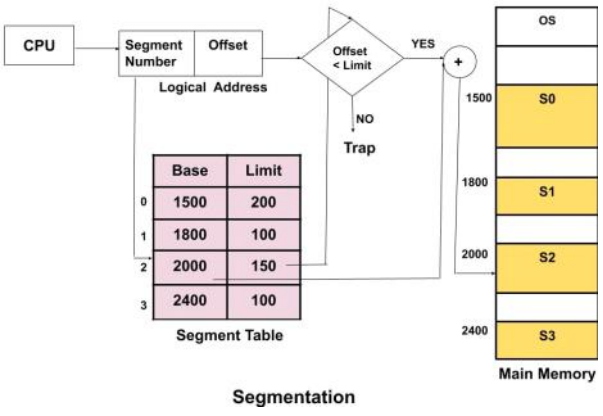
- Memory divided into **variable-size segments** based on logical units (code, stack, heap, data)
- **Segment Table:** Maps segment number to base address and limit

Logical Address = [Segment Number | Offset]
Physical Address = Base[seg] + Offset (if Offset < Limit[seg])

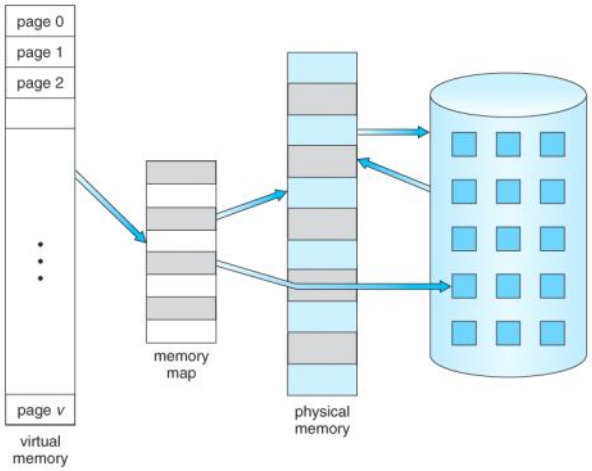
Segment	Base	Limit
0	1000	400 (Code)
1	2400	100 (Stack)
2	3200	400 (Data)

Paging vs Segmentation

Paging	Segmentation
Fixed size	Variable size
Internal fragmentation	External fragmentation
Transparent to user	User-visible (code, data, stack)
No logical grouping	Logical grouping of program



Segmentation





VIRTUAL MEMORY

Definition

- Technique that allows execution of processes **not completely in memory**
- Process's full size can be larger than physical RAM
- Creates illusion of large memory using disk storage

Demand Paging

- Pages loaded into RAM only when needed (on demand)
- **Page Fault:** Accessing a page not currently in RAM

Page Fault Handling:

1. OS detects invalid access (page fault trap)
2. Find free frame in RAM
3. Load page from disk into frame
4. Update page table
5. Restart instruction

Page Replacement Algorithms

When RAM is full and a new page needed, must **evict** an existing page:

1. FIFO (First-In First-Out)

- Replace oldest page in memory
- Simple but poor performance
- **Belady's Anomaly:** More frames can cause more page faults!

2. Optimal (OPT)

- Replace page that will not be used for longest time in future
- Theoretical best, impossible in practice (need future knowledge)
- Used as benchmark

3. LRU (Least Recently Used)

- Replace page not used for longest time in past
- Good approximation of OPT
- Implementation: Counter method or Stack method

4. LRU Approximation - Clock Algorithm (Second Chance)

- Reference bit set when page accessed
- On replacement: if reference bit = 1, clear it, give second chance
- If reference bit = 0, replace it
- Used in Linux

Example - LRU with 3 Frames:

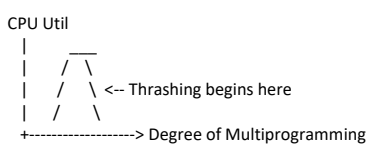
Reference String: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

Frames: [7][0][1] --> miss, miss, miss
[2][0][1] --> miss (7 replaced)
[2][0][3] --> miss (1 replaced)
... and so on

Thrashing

- **Definition:** Process spends more time paging than executing
- Happens when too many processes in memory, each getting too few frames
- Solution: **Working Set Model** -- allocate enough frames for a process's working set
- Or use **Page Fault Frequency** (PFF) control

Thrashing Curve:



SECTION 10 - FILE SYSTEM

What is a File?

- A **file** is a named collection of related information stored on secondary storage
- Types: Text, binary, source, executable, directory

File Attributes

Name, Type, Location, Size, Protection, Time/Date, Owner

File Operations

create, read, write, reposition (seek), delete, truncate, open, close

File Access Methods

Sequential Access --> Read/write one record after another (tapes)
Direct/Random --> Access any record directly by position (disks)
Indexed --> Index structure points to records (databases)

Directory Structure

Single-Level --> All files in one directory (simple, but name conflicts)
Two-Level --> Separate directory per user
Tree-Structured --> Hierarchical (used in Linux/Windows)
Acyclic-Graph --> Files shared via links (ln in Linux)
General Graph --> Allows cycles (garbage collection needed)

File Allocation Methods

1. Contiguous Allocation

All blocks of a file stored consecutively on disk
+ Fast sequential and direct access
+ Simple (just need start block + length)
- External fragmentation
- File size must be known in advance

2. Linked Allocation

Each block contains pointer to next block
+ No external fragmentation
+ File can grow dynamically
- No direct access (must traverse from start)
- Pointer overhead per block
- FAT (File Allocation Table) is a variation: pointers stored in table, not blocks

3. Indexed Allocation

One index block per file containing pointers to all data blocks
+ Direct access supported
+ No external fragmentation
- Index block overhead (wasted for small files)

Multi-level indexing (like Unix inode):
Direct blocks --> Pointers in inode to data blocks
Single indirect --> Pointer to block of pointers
Double indirect --> Pointer to block of (pointers to blocks of pointers)
Triple indirect --> One more level

Free Space Management

Bit Vector (Bitmap) --> One bit per disk block (1=free, 0=used)
Simple, fast for finding free blocks
Linked List --> Link all free blocks together
Grouping --> Store addresses of n free blocks in first free block
Counting --> Store (first_free_block, count) pairs

Inode (Index Node) - Unix/Linux

```
+------------------+
| File Metadata    | (name, size, permissions, timestamps, owner)
| Direct Blocks    | (12 direct pointers)
| Single Indirect  | (pointer to block of pointers)
| Double Indirect  |
| Triple Indirect  |
+------------------+
```

- Max file size with inodes:

If block size = 4KB, pointer size = 4 bytes
Direct: 12 * 4KB = 48KB
Single indirect: 1024 * 4KB = 4MB
Double indirect: 1024 * 1024 * 4KB = 4GB
Triple indirect: 1024^3 * 4KB = 4TB

SECTION 11 - I/O MANAGEMENT

I/O Subsystem

- OS must manage all I/O devices uniformly
- Uses **device drivers** (kernel modules specific to device)

I/O Techniques

1. Programmed I/O (Polling)
   --> CPU continuously checks device status
   --> CPU wastes cycles
   --> Simple but inefficient
2. Interrupt-Driven I/O
   --> Device sends interrupt when ready
   --> CPU does other work, handles interrupt when signaled
   --> Better CPU utilization
3. DMA (Direct Memory Access)
   --> Special hardware (DMA controller) handles data transfer
   --> CPU just initiates, DMA transfers data directly to/from memory
   --> Best: CPU free during large transfers (disks, networks)

Buffering

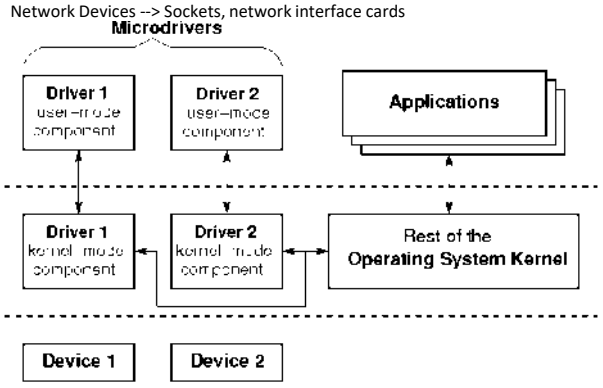
- Temporary memory area to hold I/O data
- **Single Buffering:** Process reads buffer while device fills next buffer
- **Double Buffering:** Two buffers; device fills one while process reads other
- **Circular Buffering:** Multiple buffers in ring (used in audio/video)

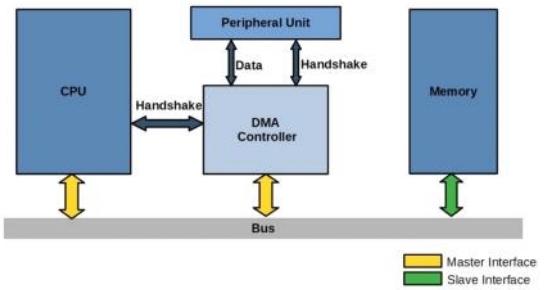
Spooling

- **Simultaneous Peripheral Operations Online**
- Data written to disk ("spool") and device reads from there
- Example: Print spooling (print jobs queued on disk, sent to printer one at a time)

Device Types

Block Devices --> Data accessed in fixed-size blocks (disks, SSDs)
Character Devices --> Data as stream of characters (keyboards, mouse, serial)
Network Devices --> Sockets, network interface cards





SECTION 12 - DISK MANAGEMENT AND SCHEDULING

Disk Structure

```
+-- Platter
|   +-- Track (concentric circles)
|   |   +-- Sector (smallest unit, typically 512B or 4KB)
|   +-- Cylinder (all tracks at same position across platters)
+-- Read/Write Head
```

Access Time = Seek Time + Rotational Latency + Transfer Time

Seek Time --> Time to move arm to correct track (dominant)
Rotational Latency --> Time to rotate disk to correct sector (~1/2 rotation avg)
Transfer Time --> Time to transfer data

Disk Scheduling Algorithms

1. FCFS (First Come First Served)

- Service requests in arrival order
- Simple but high seek time

2. SSTF (Shortest Seek Time First)

- Service request closest to current head position first
- Reduces average seek time
- **Starvation** possible for distant requests

3. SCAN (Elevator Algorithm)

- Head moves in one direction servicing all requests, then reverses
- Like an elevator going up then down
- No starvation

Example:

Head at 50, requests: 82, 170, 43, 140, 24, 16, 190
Direction: increasing

50 --> 82 --> 140 --> 170 --> 190 --> 43 --> 24 --> 16

4. C-SCAN (Circular SCAN)

- Head moves in one direction only
- When reaching end, jumps to beginning and continues (no reverse)
- More uniform wait times

5. LOOK and C-LOOK

- Like SCAN/C-SCAN but head only goes as far as last request in each direction
- More efficient than SCAN

Disk Scheduling Comparison

Algorithm	Seek Time	Starvation?	Notes
FCFS	High	No	Fair but slow
SSTF	Low	Yes	Best avg seek
SCAN	Medium	No	Good, widely used
C-SCAN	Medium	No	Uniform wait time
C-LOOK	Low	No	Most practical

SECTION 13 - STORAGE AND CACHE

Cache Memory

- Small, fast memory between CPU and RAM
- **Principle:** Locality of Reference

Temporal Locality --> Same memory location accessed repeatedly (loops)
Spatial Locality --> Nearby memory locations accessed (arrays)

Cache Levels

L1 Cache: 32-64KB, ~4 cycles (inside CPU core)
L2 Cache: 256KB-1MB, ~12 cycles (per core or shared)
L3 Cache: 4-64MB, ~40 cycles (shared across cores)

Cache Hit and Miss

Cache Hit --> Data found in cache (fast)
Cache Miss --> Data not in cache, fetched from RAM (slow)

Hit Ratio h = Cache hits / Total accesses
EAT = h * cache_time + (1-h) * (cache_time + memory_time)

Write Policies

Write-Through --> Write to both cache and memory simultaneously
Consistent but slow writes
Write-Back --> Write to cache only, update memory when block evicted
Faster writes, risk of data loss

SECTION 14 - PROTECTION AND SECURITY

Goals of Protection

- Prevent unauthorized access to resources
- Ensure each process accesses only its allowed resources

Access Matrix

Domain/Object matrix showing permissions:

	File1	File2	Printer
User1	read	read	print
User2	write	read	
User3	read	write	print

Access Control Lists (ACL)

- Each resource has list of users and their permissions
- Unix: Owner, Group, Others + Read(r), Write(w), Execute(x)

```
-rwxr-xr-- 1 alice devs 4096 Jan 1 file.txt
 |||  |||  |||
 |||  |||  +++-- Others: read only
 |||  +++------ Group: read+execute
 +++---------- Owner: read+write+execute
```

Security Concepts

Authentication --> Who are you? (password, biometric, token)
Authorization --> What can you do? (ACL, roles)
Auditing --> What did you do? (logs)

Types of Threats

Viruses --> Self-replicating malicious code
Worms --> Self-propagating across networks
Trojans --> Disguised as legitimate program
Ransomware --> Encrypts files, demands payment
Rootkits --> Hidden malware at kernel level

Ransomware --> Encrypts files, demands payment
Rootkits --> Hidden malware at kernel level

SECTION 15 - ADVANCED TOPICS

Virtualization

- **Definition:** Creating virtual versions of hardware, OS, storage
- **Hypervisor Types:**

Type 1 (Bare Metal) --> Runs directly on hardware (VMware ESXi, Hyper-V)
Type 2 (Hosted) --> Runs on top of OS (VirtualBox, VMware Workstation)

Containers vs VMs

Virtual Machines	Containers
Full OS per VM	Shared OS kernel
Heavy (GBs)	Light (MBs)
Strong isolation	Process-level isolation
Slow startup	Fast startup (seconds)
VMware, VirtualBox	Docker, LXC

Linux Completely Fair Scheduler (CFS)

- Uses **Red-Black Tree** for scheduling
- Tracks **virtual runtime** of each process
- Always picks process with smallest vruntime
- Provides fair CPU distribution

NUMA (Non-Uniform Memory Access)

- In multi-socket systems, memory access time depends on which CPU accesses which memory bank
- OS must be NUMA-aware for performance

Thrashing vs Working Set

Working Set W(t, delta) = set of pages used in most recent delta time units
If sum of all working sets > physical memory frames --> Thrashing!

SECTION 16 - OS INTERNALS SUMMARY

Boot Process

```
Power On
  --> BIOS/UEFI POST (Power-On Self Test)
  --> Bootloader found (GRUB)
  --> Kernel loaded into memory
  --> Kernel init (hardware detection)
  --> init/systemd starts (PID 1)
  --> User space services start
  --> Login prompt
```

Kernel vs User Mode

```
User Mode:
- Restricted access
- Programs run here
- Cannot directly access hardware
- Privilege Ring 3 (x86)
Kernel Mode:
- Full access to hardware
- OS runs here
- Can execute privileged instructions
- Privilege Ring 0 (x86)
Switch via: System calls, interrupts, exceptions
```

Interrupts

```
Hardware Interrupt  --> Signal from device (keyboard, disk, NIC)
Software Interrupt  --> System call (int 0x80 in Linux x86)
Exception           --> Error condition (divide by zero, page fault)
Interrupt Handling:
1. Save current state
2. Jump to Interrupt Service Routine (ISR)
3. Handle interrupt
4. Restore state
5. Resume interrupted process
```

INTERVIEW QUESTIONS - TOP 20 MOST ASKED

Q1. What is an Operating System and what are its functions?

Answer: An OS is system software that manages hardware and software resources and provides services to programs. Functions include:

- Process management (scheduling, synchronization)
- Memory management (allocation, paging, virtual memory)
- File system management

services to programs. Functions include:

- Process management (scheduling, synchronization)
- Memory management (allocation, paging, virtual memory)
- File system management
- I/O device management
- Security and protection
- Networking

Q2. What is a Process vs a Thread? When would you use threads?

Answer:

- A **process** is a program in execution with its own memory space, PCB, and resources. Heavy-weight.
- A **thread** is a lightweight unit within a process. Threads of the same process share code, data, and files but have their own stack, registers, and PC.

Use threads when:

- Tasks need to share data efficiently (no IPC overhead)
- Want responsive UIs (background thread + UI thread)
- Utilizing multi-core CPUs (parallel execution)

Q3. What is a deadlock? What are the four conditions for deadlock?

Answer: Deadlock = circular waiting where processes are permanently blocked, each holding a resource needed by another.

Four necessary conditions (Coffman):

1. **Mutual Exclusion** - Resource held by only one process
2. **Hold and Wait** - Hold resource while requesting more
3. **No Preemption** - Resources not forcibly taken
4. **Circular Wait** - Cycle in resource-allocation graph

Prevention = break any one condition. Avoidance = Banker's Algorithm.

Q4. Explain the Banker's Algorithm.

Answer: Banker's Algorithm is a **deadlock avoidance** algorithm. Before allocating resources, OS checks if the resulting state is **safe** (a safe sequence exists where all processes can finish).

Key matrices: Allocation, Max, Need (= Max - Allocation), Available. **Safe state:** If we can find an ordering of all processes where each can run to completion with available + previously allocated resources.

Q5. What is the difference between Paging and Segmentation?

Answer:

Paging	Segmentation
Fixed-size pages/frames	Variable-size segments
No external fragmentation	External fragmentation
Invisible to programmer	Programmer visible (logical division)
Internal fragmentation	No internal fragmentation
Page table per process	Segment table per process

Modern systems combine both: **Segmented Paging** (segments divided into pages).

Q6. What is Virtual Memory? How does it work?

Answer: Virtual memory allows processes to execute even when **not entirely in RAM**. It uses disk as an extension of RAM.

Working:

- Each process has a virtual address space (much larger than physical RAM)
- Only needed pages loaded via **demand paging**
- On **page fault**, OS loads page from disk (swap space) into a free frame
- **Page replacement algorithm** (LRU, FIFO, OPT) decides which page to evict

Benefits: Run large programs, more processes simultaneously, isolation.

Q7. What is a Page Fault? Explain the handling process.

Answer: A page fault occurs when a process accesses a virtual page not currently in physical memory.

Handling steps:

1. CPU traps to OS (page fault exception)
2. OS checks if reference is valid (not out of bounds)
3. Find free frame (or evict one using page replacement)
4. Load required page from disk into frame
5. Update page table (valid bit = 1, frame number)
6. Restart the faulting instruction

Q8. Explain CPU scheduling algorithms with examples.

Answer:

- **FCFS:** First come, first served. Simple, convoy effect problem.
- **SJF:** Shortest burst first. Optimal avg wait time. Starvation possible.
- **SRTF:** Preemptive SJF. Best avg wait time. High context switches.
- **Round Robin:** Time quantum. Fair, good for time-sharing. Quantum size critical.
- **Priority:** Highest priority runs. Starvation; solved by aging.
- **MLFQ:** Multiple queues with feedback. Most practical (used in Linux, Windows).

For interview: Know Gantt charts and how to calculate TAT, WT, RT.

Q9. What is the Critical Section Problem and how is it solved?

Answer: Critical section = code accessing shared resources. Problem = race conditions when multiple processes enter CS simultaneously.

Requirements: Mutual exclusion, Progress, Bounded waiting.

Solutions:

- **Hardware:** Test-and-Set, Compare-and-Swap instructions
- **Software:** Peterson's algorithm (2 processes)
- **OS:** Mutex locks, Semaphores, Monitors

Q10. Explain semaphores and the difference between mutex and semaphore.

Answer: Semaphore = integer + two atomic operations: wait() (decrement + block if < 0) and signal()

Q10. Explain semaphores and the difference between mutex and semaphore.

Answer: Semaphore = integer + two atomic operations: wait() (decrement + block if < 0) and signal() (increment + wake if ≤ 0).

Mutex	Semaphore
Binary (0 or 1)	Any non-negative integer
Ownership (locker unlocks)	No ownership (any thread can signal)
Mutual exclusion	Mutual exclusion + signaling + counting
For protecting CS	For resource counting and synchronization

Q11. What is Thrashing and how to prevent it?

Answer: Thrashing = process spends more time swapping pages than executing. CPU utilization drops sharply.

Cause: Too many processes with too few frames each. Each process page faults frequently.

Prevention:

- **Working Set Model:** Allocate frames based on working set size
- **Page Fault Frequency:** Reduce multiprogramming when fault rate too high
- **Swap out** some processes to reduce memory pressure

Q12. What is the difference between a process and a program?

Answer:

- **Program:** Static, passive. Just a file on disk containing instructions. No resources.
- **Process:** Dynamic, active. Program in execution. Has PCB, memory, CPU time, open files, I/O devices.

One program can create multiple processes (./myapp & multiple times). Same process can run same program sequentially.

Q13. What are the different types of OS scheduling (preemptive vs non-preemptive)?

Answer:

- **Non-preemptive:** Once CPU assigned to process, runs until it terminates or blocks. (FCFS, SJF)
  - Simpler, no context switches mid-run
  - Poor for interactive systems
- **Preemptive:** CPU can be taken from running process (on timer, priority change, etc.). (SRTF, RR, Priority preemptive)
  - Better responsiveness
  - Requires synchronization (shared data can be interrupted)
  - Used in modern OS

Q14. What is IPC (Inter-Process Communication)? What are its mechanisms?

Answer: IPC = mechanisms for processes to communicate and synchronize.

Types:

Shared Memory	--> Processes share a memory region; fast, needs sync
Message Passing	--> send()/receive() system calls; no shared memory needed
Pipes	--> Unidirectional byte stream between related processes
Named Pipes	--> Like pipes but between unrelated processes
Sockets	--> Communication over network or locally
Signals	--> Asynchronous notification (kill, SIGINT, SIGCHLD)
Message Queues	--> Persistent queue of messages in kernel
Semaphores	--> Synchronization primitive

Q15. What is a context switch and what is its overhead?

Answer: Context switch = saving state of current process and loading state of next process. State stored in PCB.

Overhead: Pure waste of CPU time. No useful work during switch.

- Typical time: 1-10 microseconds
- Includes: saving/restoring registers, switching memory maps, cache/TLB flush

To minimize: Use threads (cheaper switch), use larger time quantum in RR.

Q16. Explain the concept of memory-mapped files.

Answer: Memory-mapped files allow a file (or part of it) to be mapped directly into a process's virtual address space.

- File I/O happens via memory reads/writes (no explicit read/write syscalls)
- OS handles reading/writing pages from disk on demand
- Multiple processes can map the same file (shared memory via file)
- Used by: dynamic libraries (.so, .dll), database engines, mmap() in Linux

Q17. What are the differences between hard and soft real-time systems?

Answer:

Hard Real-Time	Soft Real-Time
Deadline MUST be met	Deadline desired, not mandatory
Missing = system failure	Missing = degraded performance
Safety critical	Performance critical
Airbags, pacemakers	Video streaming, games
Deterministic scheduling	Best-effort scheduling
VxWorks, FreeRTOS	Linux with RT patches, Android

Q18. What is a kernel, and what is the difference between monolithic and microkernel?

Answer: Kernel = core OS component, always in memory, runs in privileged mode.

Monolithic Kernel	Microkernel
All services in kernel	Minimal kernel (scheduling, IPC, memory)
Fast (direct calls)	Services in user space
Large code in kernel	Small trusted code base
Hard to maintain/extend	Easier to extend, more stable
Crash = full OS crash	Crash isolated to one service
Linux, Unix, Windows NT	Mach, MINIX, L4, QNX

Q19. What is the difference between fork() and exec()?

Answer:

- **fork():** Creates a new child process as a **copy** of parent. Returns 0 to child, child's PID to parent. Both continue executing from same point.
- **exec():** Replaces the current process image with a new program. Does not create new process. Old code/data replaced.

Common pattern:

```
pid = fork();
if (pid == 0) {
  exec("new_program");  // child runs new program
}
// parent continues here
```

This is how shells run commands.

Q20. What is the Dining Philosophers Problem and what does it illustrate?

Answer: 5 philosophers sit at a round table, each needing 2 chopsticks to eat. One chopstick between each pair.

Illustrates:

- **Deadlock:** All pick left chopstick simultaneously, none can get right
- **Starvation:** A philosopher might never get both chopsticks
- **Synchronization challenges** in resource sharing

Solutions:

1. Allow only 4 philosophers at a time
2. Odd philosophers pick left first; even pick right first
3. Pick both or neither (atomic)
4. Use a waiter (mutex) to grant permission

TOP 20 INTERVIEW QUESTIONS (For HW)

---

1. What is OS?

- Acts as an interface between user and hardware
- Manages resources (CPU, memory, I/O)
- Provides services to programs (file system, security)

2. Process vs Thread?

- Process: independent, separate memory space, heavy
- Thread: part of process, shared memory, lightweight
- Communication: process (IPC slow), thread (fast)

3. What is context switching?

- CPU switches from one process/thread to another
- Saves current state (PCB) and loads next process state
- Causes overhead (no actual work during switch)

4. What is scheduling?

- Mechanism to allocate CPU to processes
- Decides which process runs next
- Optimizes CPU utilization and response time

5. FCFS drawback?

- Convoy effect → small processes wait behind long ones
- Poor average waiting time
- Non-preemptive → not suitable for interactive systems

6. What is starvation?

- Low-priority process never gets CPU
- Happens in priority scheduling
- Can be solved using aging

7. What is aging?

- Gradually increases priority of waiting processes
- Prevents starvation
- Ensures fairness over time

8. What is semaphore?

- Synchronization tool using integer variable
- Controls access to shared resources
- Types: binary (0/1), counting

9. Mutex vs Semaphore?

- Mutex: locking mechanism (only 1 process allowed)
- Semaphore: counter (multiple access allowed)
- Mutex has ownership, semaphore doesn't strictly

10. Critical section problem?

- Multiple processes accessing shared resource
- Leads to race condition
- Requires synchronization (mutex/semaphore)

11. What is deadlock?

- Processes stuck waiting for each other forever
- No progress possible
- Occurs in resource sharing systems

12. Deadlock conditions?

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

(All must occur together)

13. Banker's algorithm?

- Deadlock avoidance algorithm
- Allocates resources only if system stays in safe state
- Checks safe sequence before allocation

14. Paging vs Segmentation?

- Paging: fixed-size blocks (pages), no external fragmentation
- Segmentation: variable-size logical divisions
- Paging → physical view, Segmentation → logical view

15. What is virtual memory?

- Uses disk as an extension of RAM
- Allows larger programs than physical memory
- Implemented using paging

16. What is thrashing?

- Excessive paging/swapping
- CPU spends more time in memory transfer than execution
- Caused by insufficient memory

17. What is inode?

- Stores metadata of a file (size, permissions, location)
- Does NOT store filename
- Used in Linux kernel file systems

18. File allocation types?

- Contiguous → fast but inflexible
- Linked → flexible but slow access
- Indexed → balanced (used in modern OS)

19. System call?

- Interface between user program and OS kernel
- Examples: read(), write(), fork()
- Switches from user mode → kernel mode

20. Kernel mode vs user mode?

- Kernel mode: full access to hardware
- User mode: restricted access (safe)
- Mode switch happens via system calls